ICS-344

# DVSA Vulnerability

## Discovery & Remediation

OWASP Damn Vulnerable Serverless Application on AWS


Moayd Shahat — 202277460

Osama Alghamdi — 202172210

Abdullah Alzahrani — 202265440

Rami Almathani — 201946290

# Project Overview

## What is DVSA?

**Damn Vulnerable Serverless Application**
An intentionally vulnerable serverless app by OWASP, deployed on AWS to simulate real-world security weaknesses in cloud-native environments.

### AWS Services Used:

| | |
|---|---|
| S3 | File & receipt storage |
| API Gateway | HTTP endpoint routing |
| Lambda | Serverless compute |
| DynamoDB | NoSQL order database |
| Cognito | User authentication |
| CloudWatch | Logging & monitoring |

## Investigation Goal

Conduct a structured security audit across 10 OWASP vulnerability categories on a real AWS serverless stack.

- Identify exploitable vulnerabilities
- Reproduce attacks and collect evidence
- Apply code & configuration fixes
- Verify remediation effectiveness
- Document lessons learned

# Methodology

Applied consistently across all 10 vulnerability lessons

## 01

### Identify

Analyze source code, API endpoints, IAM policies, and configuration for weaknesses

## 02

### Exploit

Reproduce the vulnerability with crafted payloads, requests, or race conditions

## 03

### Evidence

Capture CloudWatch logs, HTTP responses, screenshots, and behavior proof

## 04

### Mitigate

Apply code fixes, configuration hardening, and IAM least-privilege changes

## 05

### Verify

Re-run the attack to confirm the vulnerability is resolved; confirm secure behavior

# Vulnerability Index

| # | Lesson | Vulnerability Class | Component |
|---|---|---|---|
| L01 | Event Injection | Unsafe deserialization / RCE | DVSA-ORDER-MANAGER |
| L02 | Broken Authentication | JWT bypass / impersonation | Auth Backend |
| L03 | Sensitive Data Exposure | Unauth pre-signed URL generation | DVSA-ADMIN-GET-RECEIPT |
| L04 | Insecure Cloud Config | Weak S3 bucket controls | S3 Receipts Bucket |
| L05 | Broken Access Control | Privilege escalation on orders | Order Update Function |
| L06 | Denial of Service | No API rate limiting / throttling | API Gateway – Billing |
| L07 | Over-Privileged Function | Excessive IAM permissions | Lambda Execution Role |
| L08 | Logic Vulnerability | Race condition on order state | Billing + Update Flow |
| L09 | Vulnerable Dependencies | RCE via node-serializer 0.0.4 | DVSA-ORDER-MANAGER |
| L10 | Unhandled Exceptions | Raw stack traces leaked to client | Multiple Lambdas |

L01

# Event Injection

Component: DVSA-ORDER-MANAGER (Lambda)

| Root Cause | Exploitation | Security Impact |
| --- | --- | --- |
| Used node-serializer's unserialize() with user-controlled input in the action field — executing arbitrary JS | Crafted payload in action field triggered JavaScript execution inside Lambda; output captured in CloudWatch logs | Remote code execution (RCE) within the Lambda runtime environment |

| Fix / Mitigation | Verification | ✓ Post-Fix Result |
| --- | --- | --- |
| Replace unserialize() with JSON.parse(); validate allowed action values; reject malformed input | Injected payload returned parse error — no code execution observed in CloudWatch | **RESOLVED**<br>Injected payload returned parse error — no code execution observed in CloudWatch |

L02

# Broken Authentication

Component: Auth Backend / JWT Handler

| Root Cause | Exploitation | Security Impact |
| --- | --- | --- |
| JWT payloads decoded without signature verification — claims (username, sub) trusted without cryptographic check | Modified JWT username/sub claims to impersonate another user; backend accepted forged token as valid | Full account takeover — attacker gains access to any user's orders and data |

| Fix / Mitigation | Verification | ✓ Post-Fix Result |
| --- | --- | --- |
| Verify JWT signatures using Cognito public keys; validate iss, aud, and exp claims before trusting identity | Forged token rejected with 401 Unauthorized after fix | **RESOLVED**<br>Forged token rejected with 401 Unauthorized after fix |

L03

# Sensitive Data Exposure

Component: DVSA-ADMIN-GET-RECEIPT (Lambda)

| Root Cause | Exploitation | Security Impact |
| --- | --- | --- |
| Function generated pre-signed S3 URLs for receipt archives without verifying admin authorization | Regular authenticated user called the endpoint and received a valid pre-signed URL to download all customer receipts | Unauthorized access to sensitive customer financial receipt data |

| Fix / Mitigation | Verification | ✓ Post-Fix Result |
| --- | --- | --- |
| Add admin-role check before generating URL; restrict API routing; reduce URL expiry window | Non-admin request now returns 403 Forbidden | **RESOLVED**<br>Non-admin request now returns 403 Forbidden |

L04

# Insecure Cloud Configuration

Component: S3 Receipts Bucket

| Root Cause | Exploitation | Security Impact |
| --- | --- | --- |
| S3 bucket lacked Block Public Access and had no restrictive bucket policy; allowed direct object uploads | Uploaded a crafted file directly to S3 via pre-signed URL — bypassed API Gateway and authentication flow; triggered DVSA-SEND-RECEIPT-EMAIL Lambda | Attacker triggers backend Lambda without authentication, potentially sending malicious emails |

| Fix / Mitigation | Verification | ✓ Post-Fix Result |
| --- | --- | --- |
| Enable Block Public Access; add restrictive bucket policy; validate S3 event input in Lambda | Direct upload rejected; Lambda input validation blocks malformed events | **RESOLVED**<br>Direct upload rejected; Lambda input validation blocks malformed events |

L05

# Broken Access Control

Component: Order Update Function (Lambda)

| Root Cause | Exploitation | Security Impact |
| --- | --- | --- |
| No authorization check on the order-update endpoint; any authenticated user could trigger privileged state transitions | Regular user sent an order-update request moving order to completed/paid status, skipping billing and shipping steps | Users receive goods without completing payment workflow |

| Fix / Mitigation | Verification | ✓ Post-Fix Result |
| --- | --- | --- |
| Enforce role-based authorization; add conditional order-state validation before allowing updates | Unauthorized update attempt returns 403; order state machine enforced correctly | **RESOLVED**<br>Unauthorized update attempt returns 403; order state machine enforced correctly |

L06

# Denial of Service

Component: API Gateway – Billing Endpoint

| Root Cause | Exploitation | Security Impact |
| --- | --- | --- |
| No throttling or rate limits configured on API Gateway; billing endpoint had no concurrency controls | Sent many concurrent billing requests; multiple Lambda invocations exhausted capacity and degraded service | Service unavailability — legitimate users could not complete purchases |

| Fix / Mitigation | Verification | ✓ Post-Fix Result |
| --- | --- | --- |
| Configure API Gateway rate and burst limits; set Lambda reserved concurrency | Excess requests now return HTTP 429 Too Many Requests | **RESOLVED**<br><br>Excess requests now return HTTP 429 Too Many Requests |

L07

# Over-Privileged Function

Component: Lambda IAM Execution Role

| Root Cause | Exploitation | Security Impact |
| --- | --- | --- |
| Execution role had broad IAM policies across S3, DynamoDB, SES, and STS — far beyond function's actual needs | A compromised function could access any S3 bucket, read/write all DynamoDB tables, and assume other roles | Lateral movement and privilege escalation across the AWS account |

| Fix / Mitigation | Verification | ✓ Post-Fix Result |
| --- | --- | --- |
| Remove broad policies; grant only required SES send, specific DynamoDB table access, and CloudWatch logging | Function operates correctly; unauthorized API calls return AccessDenied | **RESOLVED**<br>Function operates correctly; unauthorized API calls return AccessDenied |

L08

# Logic Vulnerability

Component: Billing + Order Update Flow

| Root Cause | Exploitation | Security Impact |
| --- | --- | --- |
| Race condition: billing and order-update requests could run concurrently with no state lock or validation | Sent billing and update requests simultaneously; final order state differed from what was actually paid for | Order fraud — payment amount and delivered goods can be decoupled |

| Fix / Mitigation | Verification | ✓ Post-Fix Result |
| --- | --- | --- |
| Validate orderStatus before update; use conditional DynamoDB writes to block modifications once billing starts | Concurrent requests now fail safely; DynamoDB condition checks prevent inconsistent state | **RESOLVED**<br>Concurrent requests now fail safely; DynamoDB condition checks prevent inconsistent state |

L09

# Vulnerable Dependencies

Component: DVSA-ORDER-MANAGER (node-serializer 0.0.4)

| Root Cause | Exploitation | Security Impact |
| --- | --- | --- |
| Package node-serializer 0.0.4 can deserialize and execute JavaScript functions embedded in attacker input | Crafted serialized payload with embedded JS function passed to unserialize(); function was executed in Lambda | Remote code execution via supply chain — malicious npm package as attack vector |

| Fix / Mitigation | Verification | ✓ Post-Fix Result |
| --- | --- | --- |
| Remove node-serializer; replace with JSON.parse(); integrate npm audit / dependency scanning in CI | Dependency removed; payload treated as string and rejected | **RESOLVED**<br>Dependency removed; payload treated as string and rejected |

L10

# Unhandled Exceptions

Component: Multiple Lambda Functions (Python)

| Root Cause | Exploitation | Security Impact |
| --- | --- | --- |
| No try/except blocks; missing input validation — malformed requests caused unhandled exceptions | Sent invalid/missing fields; Lambda returned raw Python stack traces with file paths, line numbers, and KeyError details | Information disclosure — internal code structure exposed to attackers |

| Fix / Mitigation | Verification | ✓ Post-Fix Result |
| --- | --- | --- |
| Use event.get() for safe key access; wrap handlers in try/except; log internally; return generic error messages | Malformed requests return generic 400/500 response; stack traces no longer exposed | **RESOLVED**<br><br>Malformed requests return generic 400/500 response; stack traces no longer exposed |

# Lessons Learned

- Never trust user-controlled input — always validate and sanitize
- Always verify authentication tokens cryptographically (JWT signatures)
- Enforce authorization at every layer — not just the API gateway
- Apply least privilege to all IAM roles and execution policies
- Harden cloud resources (S3, DynamoDB, Cognito) — not only code
- Add rate limiting and throttling to serverless API endpoints
- Validate state before modification — use conditional DB writes
- Continuously scan dependencies and remove unsafe packages
- Handle errors safely — never expose stack traces or internal details